一、软件工程概述

1. 软件：程序 + 数据 + 文档

2. **软件危机**：软件开发和维护过程中，由于开发方法落后、管理不完善，软件系统复杂度不断提高而产生的一系列困难和问题，主要表现为成本增加，周期过长，质量难以保证、维护困难、开发效率低下。

   解决：技术、管理

3. 软件工程：用工程化的思想、方法、工具和管理手段来开发和维护软件，以便在规定成本和进度内交付高质量、满足用户需求的软件。

4. **软件生命周期**：

   ① 项目立项：可行性、计划

   ② 需求分析：系统做什么（功能）

   ③ 系统设计：概要设计 + 详细设计

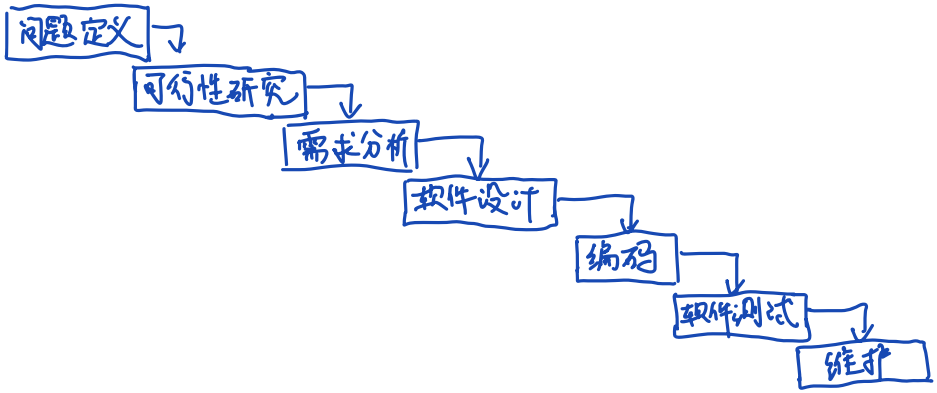
   ④ 编码：设计 → 可执行程序代码

   ⑤ 测试：验证软件

   ⑥ 维护：修复缺陷、适应环境变化，响应业务变更。

5. **软件开发生命周期模型**：

   (1) 瀑布模型（预测型模型，线性顺序模型）



按顺序进行，
阶段间验证和确认

特点：严格遵循软件生存周期各阶段的目标、任务规范与质量要求开展开发活动，强调开发前期需求定义的完备性和准确性，通过强化各阶段活动的严谨管控，力求规避开发后期修正所带来的高昂成本。

优点：提供基础框架，部分缓解早期软件危机

缺点：线性顺序模型，适应性差

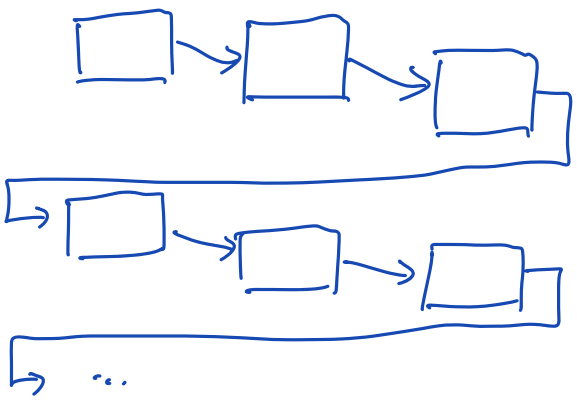
适用：简单功能，性能明确、完整，无重大需求变更

(2) 快速原型模型：

先构造一个可运行/展示的原型，让用户尽早看到系统雏形，通过反馈不断修正需求。

适用：需求不明确，需要用户反馈逐步澄清。

(3) 迭代模型/增量模型/渐增模型(适应性模型)



与瀑布区别：瀑布是整体开发模型，在下阶段开始前必须完成前一阶段所有细节；增量模型是非整体开发模型，推迟某些阶段的细节实现，较早地产生可工作的软件。

优点：快速交付，适应变化，降低风险

缺点：不适用于某些类型软件，需要更多沟通协作，系统可能不稳定

适用：复杂软件系统

(4) 喷泉模型：重叠、并行、迭代、增量、对象驱动

(5) 螺旋模型：(适应性) 瀑布+迭代+风险分析

特点：拆成多个螺旋周期，每个周期阶段似瀑布

优点：全过程风险管理，阶段质量保障，成本价值验证

缺点：高度依赖风险评估经验，迭代次数难以控

适用：复杂、高风险，需求不明确

(6) 敏捷开发：

① 个体和交互 胜过 过程和工具

② 可以工作的软件 胜过 面面俱到的文档

③ 客户合作 胜过 合同谈判

④ 响应变化 胜过 遵循计划

6. 软件开发方法：

(1) 结构化方法：顺序、选择、循环，一入口+一出口

(2) 面向数据结构方法

(3) 面向对象方法：

① 对象：基本实体，封装数据和操作的独立单元

② 封装：将对象的属性和操作绑定，隐藏对象内部实现细节

③ 继承：子类共享父类属性和操作

④ 多态：同一操作在不同子类实现不同行为

题：1. 结构化方法与面向对象方法不同点：

① 结构化方法注重控制流程和数据流程设计，主要通过顺序、选择、循环等结构完成程序的开发

② 面向对象方法将数据和方法封装在类内部，通过类的实例化来创建对象。

2. 瀑布模型、螺旋模型、敏捷开发：

① 瀑布：每个步骤依次进行，适合需求明确、简单的系统

② 螺旋：每个步骤依次进行，完成一轮迭代后进入下一轮迭代，适合需求尚不完备、需要与用户多次沟通、功能较复杂系统

③ 敏捷开发：迭代进行，但步骤间没有明确先后关系，适合用户需求变动频繁的系统。

* 项目立项（可行性分析 + 项目计划）

1. **可行性研究**：在软件项目正式启动前，从技术、经济、社会等方面系统评估项目是否能够实施，是否值得实施，为管理层是否立项提供依据。
   - 技术可行性：现有技术条件下能否实现
   - 经济可行性：经济上是否值得投资
   - 社会可行性：是否符合法律法规和社会伦理
2. 项目计划：项目范围，项目资源，进度安排，成本估算，培训计划，项目立程碑
3. 软件工作量估算：COCOMO 模型
   - 组织型：简单项目
   - 嵌入型：在紧密联系的硬件、软件和操作由限制条件下运行
   - 半独立型：介于两者之间，中等规模

基本 COCOMO：$E = a \times (KLOC)^b$

E：工作量（人月）　KLOC：千行代码

$D = c \times (E)^d$

D：开发时间（月）

生产率 $= \frac{KLOC}{E}$　　人员数 $= \frac{E}{D}$

- 某软件公司计划开发一个“企业进销存管理系统”，项目类型为半独立型。经需求分析，估算该系统的代码规模为25,000行（即25KSLOC）。采用COCOMO 基本模型进行工作量估算。
- 已知COCOMO 81基本模型的工作量计算公式为：工作量 = a × (代码行数)^b（单位：人月）

表2-1 基本COCOMO模型参数举例

方 式	*a*	*b*
组织型	2.4	1.05
半独立型	3.0	1.12
嵌入型	3.6	1.2

- （1）计算该项目的工作量（单位：人月），结果保留一位小数　$E = 3 \times 25^{1.12} \approx 110.4$ 人月
- （2）如果该项目需要投入8名开发人员，估算开发时间（单位：月），结果保留一位小数　$110.4 \div 8$
- （3）假设该公司历史项目的平均生产率为500行/人月，不考虑其他因素，按此生产率估算工作量应为多少？与COCOMO模型估算结果相比有何差异？请分析差异产生的原因。　50人月。

- 答案：（1）工作量估算
- （注意单位换算：25,000行 = 25 KSLOC，使用25代入公式，不要代入25000）
- 代入表格中半独立型公式a=3.0，b=1.12

$$E = a \times (KLOC)^b,$$

- 计算可得E=110.4人月
- （2）开发时间估算：工作量=110.4人月，团队人数N=8，因此开发时间T=110.4/8=13.8月
- （3）计算生产率：规模=25000行，生产率=500行/人月，生产率=25000/500=50人月
- COCOMO模型估算为110.4人月，生产率估算为50人月，差距产生的原因可能为：

1）项目类型因素：进销存管理系统属于半独立型，系统集成复杂度较高，需要更多沟通和协调成本，而生产率估算忽略了项目类型的影响

2）工作量组成部分：COCOMO模型估算的是项目总工作量，包括需求分析、设计、编码、测试、文档编写等全部活动；生产率估算通常仅考虑编码阶段

二、需求调查

1. 系统分析活动：收集详细信息，定义需求，优先考虑需求，开发用户界面对话，与用户评估需求。

2. 功能需求：系统能做什么，实际能让用户操作的功能

非功能需求：系统做得怎么样、表现如何，考察系统性能

用户需求：用户提出，希望系统提供哪些支持、完成哪些任务、达到哪些目标。

系统需求：细化、规范化用户需求

3. FURPS+：需求分类方法

F：Functional → 功能需求

U：Usability → 可用性：界面易用，操作简单

R：Reliability → 可靠性：故障率、故障恢复

P：Performance → 性能：响应时间、吞吐率、并发量

S：Security → 安全：访问控制、加密

+：其他约束

4. 模型与建模：建模可以通过抽象降低系统复杂性，帮助分析人员理解需求，记录系统细节，促进开发人员、用户和利益相关者之间沟通，并为后续维护和功能增强提供依据

5. 利益相关者：

按组织 {
- 内部利益相关者：组织内部人
- 外部利益相关者：组织外部人，可影响项目或被项目影响

按交互 { 操作利益相关者：直接与系统交互的人
执行利益相关者：不直接操作系统，但对系统有各种影响

在医院信息管理系统中，临床护士属于内部操作利益相关者。

护理部主任、医院IT部门主管、院长均属于内部执行利益相关者。

患者（或其家属）通过移动端应用查看检验报告、预约挂号、在线咨询，属于外部操作利益相关者。

卫生健康委员会是典型的外部执行利益相关者

例子：上市公司会计系统的利益相关者

内部-操作相关者：出纳，会计，内部审计师
内部-执行相关者：高级经理，董事会
Both：业务经理

外部-操作相关者：合作伙伴组织；客户
外部-执行相关者：投资者，监管机构
Both：外部审计师

6. 活动图

7. 结构化分析：

(1) 数据流图：顶层 □→0→□，0层

- 数据流图

表 3-1 数据流图中的基本图形符号

符号	说明
○	圆或椭圆。表示加工，也称数据处理，它对数据流进行某些操作或变换。应在圆圈内写上加工名字（一般为动词短语），在分层数据流图中，加工还应编号
□	矩形。表示数据流的源点和终点，是软件系统外部环境中的实体（包括人员、组织或其他软件系统），统称外部实体。应在方框内写上相应的名称
═	双杠。表示文件或称数据存储，暂时保存数据，它可以是数据库文件或任何形式的数据组织。指向存储的数据可理解为写入，从存储引出的数据可理解为读出。要用名词或名词短语进行命名
→	箭头。表示数据流，由一组成分固定的数据项组成。数据流必须有流向，箭头表示数据流动的方向，即在加工之间、加工与源点/终点之间、加工与数据存储之间流动。除与数据存储之间的数据流不用命名外，其他数据流均需用名词或名词短语命名

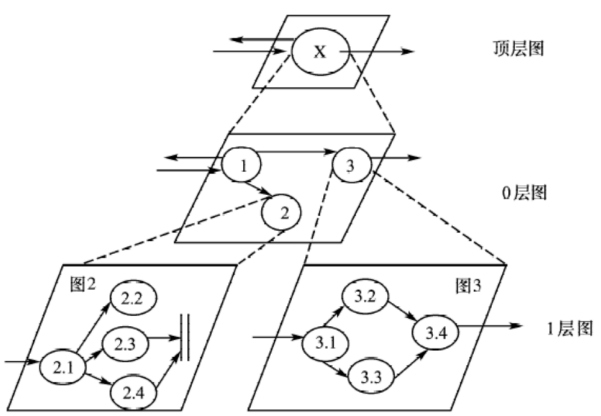


图 3-2 系统分层数据流图

(2) 数据字典：名字、别名/编号、分类、内容描述、何处使用

表 3-2　　数据词典中常用符号

符号	含义	举例及说明
=	定义为	
+	与，连接	x=a+b 表示 x 由 a 和 b 组成
[… | …]	选择	x=[a|b|c]表示 x 由 a 或 b 或 c 组成，如：性别=[男|女]，表示性别是“男”或“女”之一
{… }	重复	x={a}表示 x 由 0 个或多个 a 组成，如：姓名=1{汉字}4，表示姓名为 1 至 4 个汉字；又如：1{字母}，表示至少 1 个字母；{字母}，表示 0 至任意个字母
(…)	可选	x=(a)表示 a 可在 x 中出现，也可不出现
..	连接符	x=1..9 表示 x 可取 1 到 9 任意一个值
* … *	注释	表示两个 * 之间的内容是对条目的注释

考虑算我侧窗

(3) 结构化语言：

表 3-3　　结构化语言

	书写格式	含义
选择结构	IF〈条件〉 THEN〈策略〉	如果〈条件〉 则〈策略〉
	IF〈条件〉 THEN〈策略 1〉 ELSE〈策略 2〉	如果〈条件〉 则〈策略 1〉 否则〈策略 2〉
	按下列条件选择策略 CASE〈条件 1〉〈策略 1〉 CASE〈条件 2〉〈策略 2〉 … CASE〈条件 N〉〈策略 N〉	按下列条件选择策略 情况〈条件 1〉〈策略 1〉 情况〈条件 2〉〈策略 2〉 … 情况〈条件 N〉〈策略 N〉
循环结构	WHILE〈条件〉DO 〈策略〉 END DO	每当〈条件〉成立则 〈策略〉 返回
	REPEAT 〈策略〉 UNTIL〈条件〉	重复以下操作 〈策略〉 直到〈条件〉

(4) 判定表：

某数据流图中有一个“确定保险类别”的加工，指的是申请汽车驾驶保险时，要根据申请者的情况确定不同的保险类别。加工逻辑为：如果申请者的年龄在21岁以下，要额外收费；如果申请者是21岁以上并是26岁以下的女性，适用于A类保险；如果申请者是26岁以下的已婚男性，或者是26岁以上的男性，适用于B类保险；如果申请者是21岁以下的女性或是26岁以下的单身男性，适用于C类保险。除此之外的其他申请者都适用于A类保险。

（1）提取问题中的条件。条件是年龄、性别及婚姻。

（2）标出条件的取值。如表3-3所示，用符号代替条件的取值。

条件名	取 值	符 号	取值数m
年龄	年龄≤21 21＜年龄≤26 年龄＞26	C Y L	$m_1 = 3$
性别	男 女	M F	$m_2 = 2$
婚姻	未婚 已婚	S E	$m_3 = 2$

（3）计算所有条件的组合数N：

$$N = \prod_{i=1}^{3} m_i = 3 \times 2 \times 2 = 12$$

（4）提取可能采取的动作或措施。这些措施要适用于A类保险、B类保险、C类保险和额外收费。

（5）制作判定表

判 定 表

	1	2	3	4	5	6	7	8	9	10	11	12
年龄	C	C	C	C	Y	Y	Y	Y	L	L	L	L
性别	F	F	M	M	F	F	M	M	F	F	M	M
婚姻	S	E	S	E	S	E	S	E	S	E	S	E
A类保险					√	√			√	√		
B类保险				√				√			√	√
C类保险	√	√	√				√					
额外收费	√	√	√	√								

(5) 判定树

要素	说明
根节点	决策的起点，第一个需要判断的条件
内部节点	中间的条件判断点
分支	从节点出发的边，代表条件的取值
叶子结点	决策的重点，表示执行的操作或结论

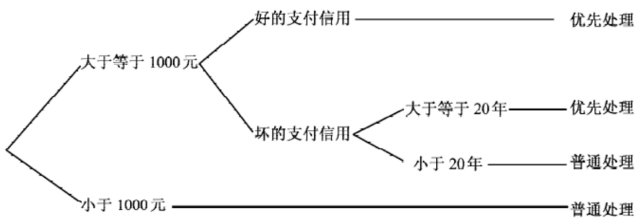


图 3-8　图书优惠政策判定树

三、用例：

1. 用例：面向对象需求分析中的核心概念，用于描述系统在外部参与者的交互下所执行的完整功能单元。

命名：动词+名词

判断用例粒度：基本业务流程 EBP

需求识别方式：① FURPS+ ② 事件分解 ③ CRUD方法

2. 识别用例：用户目标技术，事件分解技术

找用户，问目标，确定用例　　系统必须响应哪些事件

事件：
- 外部事件：系统外部发生
- 时间事件：到达某时间点触发
- 状态事件：系统内部状态变化触发

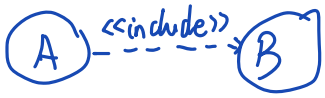
理想技术假设：认为设备不会损坏，处理和存储能力没有限制，用户完全遵照系统要求且没有误操作。只专注于找用户工作的功能。

3. CRUD技术：（增删改查）

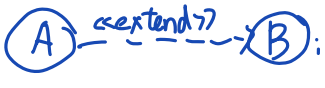
C：Create　R：Read/Report　U：Update　D：Delete

用于验证、改进或交叉检查用例

4. 用例图：



：做A必须做B



：做B可能做A



：A是B的一种

四、域建模

1. 问题域中的事物就是对象 → 域类
2. 识别域类：头脑风暴法、名词技术
   - 头：先选定用户和用例，然后和用户一起讨论，得出执行用例时系统需要记住的事物
   - 名：找名词，判断能否成域类
3. E-R图、域模型类图

五、扩展需求模型

1. 用例描述：用例名称、参与者、前置条件、后置条件、基本流程、备选流程、业务规则、场景、触发事件、简要描述、利益相关者

   前置条件：用例开始前必须满足的条件

   后置条件：用例完成后系统必须达到的状态

   活动流程：actor：～ system：～
2. 用例活动图：活动图
3. 系统顺序图SSD：actor + :System

   ⟶ 系统输入请求　　⟵ 系统返回输出

   *：循环发生　　[条件]：判断
4. 状态机图：画某个对象生命周期

   转换：事件 [保护条件]/动作

六、设计要点与设计活动

1. **概要设计**：系统整体结构和组织方式，架构设计

   详细设计：每个模块数据结构、算法、测试用例、接口、用户界面

2. 设计原理：模块化、抽象、信息隐蔽、逐步求精、模块独立性（高内聚低耦合）

3. **内聚**：衡量一个模块内部各元素彼此结合的紧密程度。内聚↑ 紧密↑

低↓

① 巧合性内聚：模块内部没关系

② 逻辑性内聚：把几种逻辑相关功能放一起，由参数决定（flag）

③ 时间内聚：模块中任务必须在同一时间段执行

④ 过程内聚：模块内处理元素必须按特定次序执行

⑤ 通信内聚：元素用同一输入或产生同一输出

⑥ 顺序内聚：一个处理元素的输出是另一个的输入

⑦ 功能内聚：模块内所有元素共同完成单一功能

高

4. **耦合**：对软件结构内不同模块之间互连程度的衡量。

低↓

① 无直接耦合：两模块间无直接关系

② 数据耦合：通过参数传递数据，且只传普通数据

③ 标记耦合：传数据结构的一部分或整体，模块只用其中一部分

④ 控制耦合：传控制信息，影响被调用模块执行哪条逻辑

⑤ 外部耦合：多模块依赖同一外部接口、设备、格式、协议

⑥ 公共耦合：多模块共享同一全局数据环境

⑦ 内容耦合：一个模块直接访问或修改另一个的内部内容

高

尽量使用数据耦合，少用控制耦合，限制公共环境耦合范围，不用内容耦合

5. 启发式设计规则:

① 改进软件结构,提高模块独立性

② 模块规模适中

③ 深度(控制层数)、宽度(同层模块数)、扇入(被调)、扇出(调用)

④ 模块作用域在控制域之内

6. 设计活动:

① 设计环境　　⑤ 设计应用程序结构和软件

② 设计系统界面　　④ 设计用户界面

③ 设计数据库　　⑥ 设计系统控制和安全

7. 系统架构:概要设计的核心任务

(1) 集中式架构:所有计算任务和处理功能都集中在一个中心节点上,客户端仅负责输入输出和结果显示。

(2) 分布式架构:系统的不同组件分布在多个独立的计算机节点上,节点之间通过网络进行通信和协作,共同完成系统功能

(3) 分层架构:系统按功能层次组织为若干层,每层提供特定服务,并为上层提供接口。

三层架构:

① 表示层:用户界面、交互逻辑

② 业务逻辑层:业务规则、数据处理

③ 数据访问层:数据库操作、数据存取

8. Internet 部署的托管替代方案：

① 场地出租：主机托管，提供安全位置，但计算机由客户企业拥有

② 管理服务：客户拥有软件，但可能购买额外服务

③ 虚拟服务器：客户租用配置好的虚拟服务器，拥有一定CPU、内存、磁盘、带宽

④ 云计算：虚拟服务器的扩展，资源看起来有近乎无限的可用性

↳ 客户应像购买水电一样的买计算能力，不需关心算力如何/在哪提供。

9. 概要设计原则：

① 关注点分离原则

② 高内聚、低耦合原则

③ 可扩展性原则

④ 可重用性原则

10. 架构设计原则：

① 最小知识原则：每个模块应尽可能少地了解其他模块的内部细节，只通过公开接口进行交互

② 渐进式设计原则：架构设计应支持增量开发和迭代交付，能够分阶段实现系统功能

七、界面设计

1. 用户界面：直接涉及人类用户或参与者的输入和输出。

系统界面：输入输出需要很少人工干预的接口，包括自动捕获输入、直接输出到其他系统、打印和分发输出等。

2. 用户中心设计：从用户角度看，用户界面就是整个系统，界面背后的程序、数据库、硬件对用户来说不重要。

原则：① 及早关注用户及其工作

② 评价设计以确保可用性

③ 使用迭代开发方法

3. HCI (Human-Computer Interaction)：关注用户与计算机交互的效率、有效性、输入输出技术和心理因素。

可视性：控件必须被用户看见

提示性：控件外观暗示其功能

4. 通用用户界面设计原则：一致性、快捷方式、反馈、完整对话、错误处理、易撤消、减轻短期记忆负担

5. 系统界面：与其他系统的输入输出，高度自动化输入输出、外部数据库输入输出

6. 系统输入：无错误、降人工

7. 系统输出：

① 详细报告：具体业务交易信息

② 摘要报告：汇总细节或回顾周期性活动

③ 异常报告：显示超出正常范围的事务或操作结果

④执行报告：管理者用于评估整体状态和绩效

⑤内部输出：组织内部使用的报告或输出

⑥外部输出：供组织外部人员使用的报告或文档

⑦返回文档：外部输出中包含一部分需要带新数据返回

8. 人机交互 设计原则：可用性、反馈原则、容错性、效率原则、一致性

八、系统开发方法

1. SDLC 系统开发生命周期：预测方法（瀑布）、适应方法（迭代）
2. 支持：
   ① 维护系统：处理系统运行后的问题
   ② 增强系统：添加/改变功能
   ③ 支持用户：持续用户培训/帮助台
3. 结构化方法：系统 = 过程 + 数据流
   - 结构化分析：数据流图、数据字典
   - 结构化设计：结构图
   - 结构化程序设计：程序流程图、N-S图、PAD图

   面向对象方法：系统 = 相互协作的对象集合
   - OOA：用例图、域模型类图、系统顺序图、状态机图
   - OOD：类图、顺序图、控制器、三层结构
   - OOP：实现类和对象
4. 结构图

十、面向对象设计原则

1. 类图：

(1) 设计类的构造型：

① 实体类 «entity»

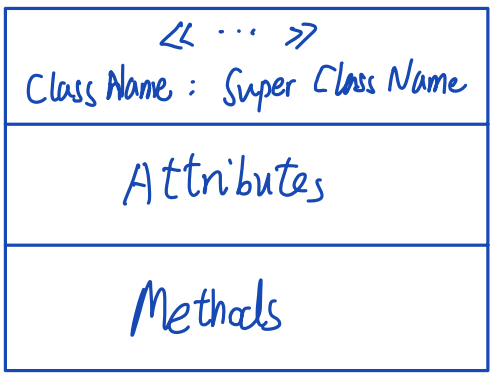
② 持久类：系统关闭也存在

③ 边界类/初图类：«boundary» 位于系统边界，如输入，Web等

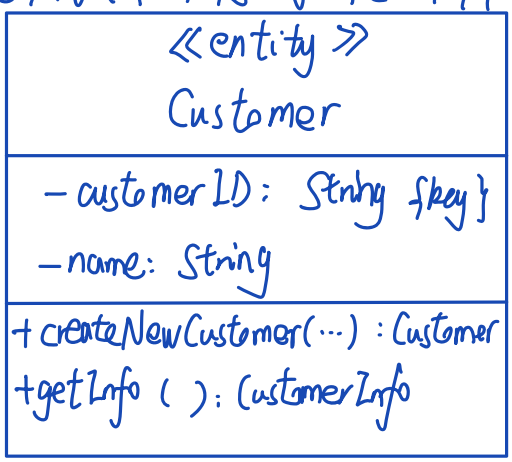
④ 控制类：«control» 在边界类和实体类间起协调作用，从边界类获取信息发给实体类

⑤ 数据访问类：«data access» 从数据库获取信息或向数据库发送信息

(2) 类：



例：



(3) 属性：(+/-) name : type (= initial value) ({key})

(4) 方法：(+/-) name (parameter) : returnType

(5) 类级属性/类级方法 (static)：加下划线

(6) 抽象类：不能实例化，类名斜体

(7) 导航可见性(→)：一个对象查看另一对象

2. CRC卡：Class + Responsibility + Collaborator

3. 控制器类(Controller): 在 Boundary 与 Entity 之间，降低耦合

4. 面向对象详细设计原则:

① 单一职责原则 SRP: 一个类应该只有一个引起它变化的原因

② 开闭原则 OCP: 对扩展开放，对修改关闭

③ 里氏替换原则 LSP: 子类必须能够替换父类

④ 接口隔离原则 ISP: 客户端不应被迫依赖它不需要的接口

⑤ 依赖倒置原则 DIP: 高层模块不依赖低层模块，二者依赖抽象

⑥ 最少知识原则 LoD: 一个对象应尽量少了解其他对象。

十一、面向对象设计：用例实现

1. 顺序图

Actor → View → Controller → Domain → Data Access

└ 格式：对象名：类名

2. 协作图：

[true/false condition] sequence_number : return_value := message_name (parameters)

3. 设计模式：

(1) Controller 控制器模式：使视图层和域层之间更松耦合

(2) Adapter 适配器模式：接口不兼容，但可一起工作

(3) Singleton 单例模式：一个类只有一个实例

(4) Factory 工厂模式：批量生产工具类

十二、系统实现

1. 软件重用：知识重用、方法和标准的重用、软件成分的重用

软件成分重用：代码重用、设计结果重用、分析结果重用

2. 程序设计风格：提高可重用性

① 提高方法内聚

② 减小方法规格

③ 保持方法一致性

④ 把策略和实现分开

策略方法：做决策、管理全局、检查状态、处理错误

实现方法：完成具体操作或算法，不决定是否执行，不管理全局

⑤ 全面覆盖

⑥ 尽量不使用全局信息

3. 程序设计风格：提高健壮性

① 预防用户操作错误

② 检查参数合法性

③ 不要预先确定限制条件

④ 先测试后优化

4. 实现顺序：

自顶向下 — stub：模拟尚未开发完成的被调用模块（下级被调用）

自底向上 — driver：调用被测模块，传入参数，接收返回结果（上级调用者）

SCCS：跟踪源代码并控制变更

5. 测试：

① 单元测试：测试单个模块、方法或类（白盒）

② 集成测试：测试一组模块或方法（白/黑）

③ 系统测试：测试整个系统的行为（黑盒）

④ 可用性测试，性能测试，验收测试

十三、测试策略

1. 测试：在交付最终用户之前，以发现错误为特定目的来运行程序的过程（错误/需求/性能/指标）
2. Verification 验证：设计、编码是否符合规格
   Validation 确认：需求是否被满足
3. 测试顺序：单元测试 → 集成测试 → 确认测试 → 系统测试
4. 单元测试：测单个模块/方法/类
   - 检查：接口、局部数据结构、边界条件、独立路径、错误处理路径
   - → 白盒
5. Driver 驱动模块：模拟上级调用者（↑）
   Stub 存根模块：模拟下级被调用模块（↓）
6. 集成测试：关注构件开始相互作用时，与验证和程序构造相关的问题。
   - → Big-bang Testing：所有模块一次性集成（纯废物）
   - → Top-down Integration：从主控开始，从上到下，下层用 stub
   - → Bottom-up Integration：从底层开始向上，上层用 driver
   - → Sandwich Testing：折中，两头测试，中间汇合
7. 回归测试：修改后重测，避免引入新错误
8. 冒烟测试：每次 build 后做一组基础关键测试，确认系统无大问题
9. 通用软件测试准则：
   ① 接口完整性：每个模块或模块集加入系统时，测试内/外模块接口
   ② 功能有效性：测试以发现软件功能缺陷

③信息内容：测试局部或全局数据结构中的错误

④性能：测试规定的性能边界

10. OO 测试：

- 单元测试：类测试
- 集成测试：

① 基于线程的测试：测响应一个事件所需所有类

② 基于使用的测试：先测独立类，再测依赖这些类的类

③ 集群测试：测试一组协作类的交互错误

11. 确认测试：测软件是否满足需求

12. 系统测试：

① 恢复测试：强制软件失败，测恢复能力

② 安全测试：验证保护机制是否能防止非法侵入或数据篡改

③ 压力测试：检查系统面对异常资源需求时的表现

④ 性能测试：测试运行时性能

13. 调试：强力法、回溯法，归纳法、演绎法

十四、测试方法　　你已胜利！

1. 好的测试：尽可能全面地发现错误
2. **白盒测试**：知道程序内部结构，根据程序逻辑设计测试
   - **逻辑覆盖法**：（关注程序流程图判定框）
     ① 语句覆盖：每条可执行语句至少执行一次（假分支不执行）
     ② 判定覆盖：每个判断的真假分支至少执行一次
     ③ 条件覆盖：每个条件（T/F）至少出现一次（不保证分支都执行）
     ④ 判定/条件覆盖：同时满足判定覆盖和条件覆盖
     ⑤ 条件组合覆盖：每个判断中所有条件组合至少执行一次
   - 路径测试：测试独立路径（至少包含一条以前路径没有的新边）
     环路复杂度：[独立路径数 = 环路复杂度 = 测试用例数]
     ① 边(E)、节点(N)，$V(G)=E-N+2$
     ② 判定节点数(P)，$V(G)=P+1$
     ③ $V(G)$ = 区域数
   - 循环测试：
     ① 简单循环：设循环最大次数n，应测0、1、2、m、n-1、n、n+1次
     ② 嵌套循环：先测最内层，再逐层向外
     ③ 串接循环：分别按简单循环进行顺序测试
     ④ 非结构化循环：避免

3. 黑盒测试：不关心程序内部逻辑，只根据功能说明和输入输出关系设计测试用例

(1) 等价类划分：

例：16≤age≤35

① 有效等价类：符合规格说明的输入集合 → 16~35

无效等价类：不符合规格说明的输入集合 → <16，>35

② 划分规则：

- 输入范围：1~999

→ 无效类：x<1　有效类：1≤x≤999　无效类：x>999

- 输入集合：红、绿、蓝

→ 有效类：红、绿、蓝　无效类：其他颜色

- 布尔值：是否会员

→ 有效类：true　无效类：false

- 格式要求：手机号11位数字

→ 有效类：11位数字　无效类：长度不足

无效类：长度过长　无效类：不是数字

③ 设计用例：每个无效等价类至少一个测试用例

例如输入 `a` 范围为 `(-100,0)`，输入 `b` 范围为 `(0,100)`：

```
a 的有效类：-100 < a < 0
a 的无效类：a ≤ -100
a 的无效类：a ≥ 0

b 的有效类：0 < b < 100
b 的无效类：b ≤ 0
b 的无效类：b ≥ 100
```

测试用例可以设计为：

用例	a	b	覆盖
1	-50	50	a、b 均有效
2	-100	50	a 无效：a ≤ -100
3	0	50	a 无效：a ≥ 0
4	-50	0	b 无效：b ≤ 0
5	-50	100	b 无效：b ≥ 100

(2) 边界值分析：测试数据在边界及附近

例：范围 $a \le x \le b$

测：$a-1, a, a+1, b-1, b, b+1$

输出、内部数据结构亦有边界

(3) 错误推测法：依靠直觉和经验

(4) 因果图法